

O dataset da Olist é um excelente laboratório para simular desafios reais de e-commerce, especialmente pela pegadinha arquitetônica de separar o `customer_id` (que é gerado a cada pedido) do `customer_unique_id` (que identifica a pessoa física real).

Abaixo, apresento o design completo do sistema end-to-end para o modelo de **Propensão de Recompra (Churn / Retention)**, unindo rigor técnico de engenharia de dados com impacto direto em métricas de negócio.

1. OBJETIVO DO MODELO

O Problema de Negócio:

No e-commerce, o Custo de Aquisição de Clientes (CAC) é historicamente alto devido à forte competição em mídia paga. Por outro lado, reter um cliente custa muito menos. O problema é que o churn em e-commerce não é explícito (o cliente raramente vai lá e cancela a conta; ele simplesmente para de comprar).

O Objetivo Técnico:

O modelo será focado na **Classificação e Geração de Score de Propensão de Recompra**.

- **Por que não apenas regressão (dias para próxima compra)?** Porque a regressão sofre muito com clientes que *nunca* voltam, gerando ruído e outliers.
- **A abordagem ideal:** Um modelo de classificação binária que cospe uma probabilidade contínua (Score de 0 a 100%). Isso permite que o time de CRM crie réguas de relacionamento (ex: dar desconto de 20% para quem tem score entre 30-50%, mas não dar desconto para quem tem >90% pois ele já iria comprar de qualquer forma).

2. DEFINIÇÃO DO TARGET (SAÍDA DO MODELO)

Para definir o target, usaremos uma abordagem de **Snapshot Framing**. Vamos definir datas de observação (safras) e olhar para o futuro para ver se o cliente comprou.

- **Target:** `is_retained_90d` (Binário: 1 ou 0).
- **Mecânica:** Fixamos uma data de referência (ex: 1º de Janeiro). Olhamos para todos os clientes que compraram nos últimos 12 meses. O modelo deve prever: *Este cliente fará uma nova compra nos próximos 90 dias (Jan-Mar)?*
- **Prevenção de Data Leakage (Vazamento de Dados):**
 - Se a data de referência é `$T_0$`, **absolutamente nenhuma feature** pode usar dados de pedidos onde `order_purchase_timestamp > $T_0$`.
 - Reviews de compras passadas feitas *depois* de `$T_0$` não podem ser usadas.

3. ARQUITETURA DE DADOS E JOINS

A granularidade final (a "Tabela Fato" do modelo) será em nível de **customer_unique_id + data_de_referencia**.

Fluxo de Construção (Camadas):

1. Base: Conectar orders com customers para descobrir o customer_unique_id real de cada pedido.
2. Expandir Pedido: Fazer JOIN de orders com order_items e order_payments.
3. Enriquecer Produtos/Sellers: Trazer products (e categorias em inglês) e sellers.
4. Feedback e Logística: Trazer order_reviews e calcular tempos baseados nas datas da orders.

Fluxo Lógico (Diagrama):

Plaintext

[olist_customers] (Chave central de identificação)

↑ (customer_id)

[olist_orders] (Tabela Fato Transacional)

↓ (order_id)

├── [olist_order_items] → (product_id) → [olist_products] → [category_translation]

├── ↓ (seller_id)

├── [olist_sellers]

├──

├── [olist_order_payments]

├──

└── [olist_order_reviews]

Geolocalização ([olist_geolocation]) corre por fora, unida via `zip\_code\_prefix` dos clientes e sellers.

4. FEATURE ENGINEERING (MUITO DETALHADO)

Criaremos agregações olhando apenas para o período *anterior* a \$T_0\$.

A. Features de Cliente (Comportamental / RFM)

- `recency_days`: Dias desde a última compra até \$T_0\$.
- `frequency_total`: Número total de pedidos efetivados.
- `monetary_total_spent`: Soma total de `payment_value`.
- `monetary_ticket_medio`: Valor médio por pedido.
- `trend_monetary`: Razão entre o gasto do último pedido e a média dos anteriores (identifica se está gastando mais ou menos).
- `avg_items_per_order`: Quantidade média de itens na cesta.

B. Features de Logística (Atrito Operacional)

- `avg_delivery_time_days`: Média de dias reais de entrega (`delivered_customer_date` - `purchase_timestamp`).
- `delay_rate`: % de pedidos onde a entrega real superou a `estimated_delivery_date`.
- `freight_to_price_ratio`: Média do (valor do frete / valor do pedido). Fretes proporcionalmente muito caros geram atrito para a recompra.
- `avg_distance_km`: Distância média de Haversine entre o `customer_zip_code` e os `seller_zip_code`.

C. Features Financeiras

- `preferred_payment_type`: Moda (mais frequente) do tipo de pagamento (Credit Card, Boleto, Voucher).
- `avg_installments`: Média de parcelas. Clientes com parcelas muito longas ativas podem demorar mais a recomprar.
- `boleto_usage_rate`: % de pedidos pagos via boleto.

D. Features de Produto

- `distinct_categories_bought`: Número de categorias únicas (indica penetração transversal no catálogo).
- `most_frequent_category`: Categoria principal comprada.
- `avg_product_volume_cm3`: Volume médio dos pacotes recebidos (produtos pesados = móveis/eletro; produtos leves = moda/beleza, que têm mais recorrência).

E. Features Temporais

- `last_purchase_month`: Mês da última compra (capta sazonalidade).
- `is_holiday_shopper`: Booleano se a maioria das compras ocorreu em Novembro/Dezembro (Black Friday/Natal). Clientes assim só voltam no ano seguinte.

F. Features de Engajamento/Satisfação

- `avg_review_score`: Média das notas de 1 a 5. (Esta é a feature mais preditiva do churn).
- `silent_customer_rate`: % de pedidos em que o cliente não deixou review (indiferença).
- `negative_reviews_count`: Quantidade de notas 1 e 2.
- *Extra*: Contagem de caracteres do `review_comment_message`. Reviews longas e negativas são indicadores fortíssimos de churn absoluto.

5. MODELAGEM

Para dados tabulares heterogêneos com variáveis categóricas, ausência de dados (nulos) e relações não-lineares, modelos baseados em árvores são absolutos.

- **1º Lugar: LightGBM (Recomendado).**
 - *Vantagens*: Extremamente rápido, lida nativamente com nulos, suporta categorias sem precisar de One-Hot Encoding prévio (economiza memória). Ótimo desempenho.
 - *Desvantagens*: Pode sofrer overfitting em datasets minúsculos (o que não é o caso aqui).
- **2º Lugar: CatBoost.**
 - *Vantagens*: Lida com variáveis categóricas melhor que qualquer outro, precisão absurda out-of-the-box.
 - *Desvantagens*: Treinamento um pouco mais lento que o LightGBM.
- **3º Lugar: XGBoost.**
 - *Vantagens*: Padrão da indústria, robusto.
 - *Desvantagens*: Requer encoding manual rigoroso e mais pré-processamento.
- *Rejeitados*: Redes Neurais (requerem normalização intensa, sofrem com features esparsas e oferecem pouca interpretabilidade de negócio) e Regressão Logística (não capta não-linearidades do RFM).

6. PIPELINE DE MACHINE LEARNING

1. **Ingestão**: Leitura dos CSVs/Banco particionado via Polars (mais rápido que Pandas).
2. **Limpeza & Nulos**:
 - Datas ausentes em `delivered_date` (ex: pedidos cancelados) recebem um marcador ou valor limite.
 - Reviews sem texto recebem "SEM_COMENTARIO".

3. **Feature Engineering:** Criação das agregações com *Window Functions* olhando para trás no tempo.
4. **Encoding:** Label Encoding/Ordinal Encoding para categorias se usar XGBoost. Se usar LightGBM, apenas converter a coluna para o tipo category.
5. **Treinamento (OOT):** Treino na Safra 1, Validação na Safra 2.
6. **Tuning:** Otimização Bayesiana com Optuna buscando maximizar a métrica de negócio.
7. **Registro:** Logging de hiperparâmetros, artefatos do modelo e métricas via MLflow.
8. **Deploy:** Empacotar modelo para Batch Scoring diário via Airflow.

7. ESTRATÉGIA DE TREINO (EVITANDO DATA LEAKAGE)

Nunca faça o clássico `train_test_split` (divisão aleatória). Em e-commerce, **o tempo importa**. Você deve usar **Out-of-Time (OOT) Validation**.

- **Treino:** Comportamento de clientes até Outubro/2017. Target: comprou em Nov/Dez/Jan?
- **Validação:** Comportamento até Janeiro/2018. Target: comprou em Fev/Mar/Abr?
- **Teste (Holdout):** Comportamento até Maio/2018. Target: comprou em Jun/Jul/Ago?

Isso prova que o modelo consegue generalizar para safras futuras do mundo real, sobrevivendo a mudanças sazonais e econômicas.

8. MÉTRICAS IDEAIS

Como as classes serão desbalanceadas (a maioria dos clientes *não* recompra em 90 dias):

- **Esqueça a Acurácia.** Se 90% não recompra, um modelo "burro" que prevê 0 para todo mundo tem 90% de acurácia e 0 valor de negócio.
- **Para o Cientista de Dados:**
 - **PR-AUC (Precision-Recall Area Under Curve):** Excelente para classes desbalanceadas.
 - **ROC-AUC:** Mede a capacidade geral de ordenação (separar bons de ruins).

- **Para o Negócio (Cruciais):**
 - **Lift (Decile Analysis):** "Se eu pegar os 10% de clientes com maior score, quantas vezes melhor eu sou em achar recompradores em comparação a um sorteio aleatório?"
 - **KS (Kolmogorov-Smirnov):** Mede a separação máxima entre a curva de quem recompra e quem não recompra.

9. INSIGHTS DE NEGÓCIO

- **Marketing (CAC vs LTV):** Direcionar anúncios de retargeting no Facebook/Google Ads APENAS para os deciles 2, 3 e 4 (clientes na dúvida). Não gastar dinheiro com o decil 1 (já vão comprar) nem com o 10 (causa perda/haters).
- **Logística:** Se o SHAP value mostrar que `delay_rate` e `freight_to_price_ratio` são os maiores ofensores de churn, a empresa tem embasamento matemático para subsidiar frete ou penalizar transportadoras/sellers ruins.
- **CRM:** Criar gatilhos automatizados. Cliente fez compra, a nota de review foi 2, e o score de propensão despencou? Aciona um humano do SAC no mesmo dia para tentar reverter a experiência ruim com um voucher.

10. OUTPUT FINAL ESPERADO

Pseudocódigo de Treinamento:

```
Python
import polars as pl
import lightgbm as lgb
from sklearn.metrics import roc_auc_score
import shap

# 1. Carregar Dados de Safra (OOT Split)
# df_train = Safra Passada, df_test = Safra Futura
X_train = df_train.drop(["customer_unique_id", "target_recompra_90d"])
y_train = df_train["target_recompra_90d"]

X_test = df_test.drop(["customer_unique_id", "target_recompra_90d"])
y_test = df_test["target_recompra_90d"]
```

2. Configurar Modelo

```
params = {  
    'objective': 'binary',  
    'metric': 'auc',  
    'learning_rate': 0.05,  
    'num_leaves': 31,  
    'class_weight': 'balanced', # Lida com desbalanceamento  
    'random_state': 42  
}
```

3. Treinamento

```
model = lgb.LGBMClassifier(**params)  
model.fit(  
    X_train, y_train,  
    eval_set=[(X_test, y_test)],  
    callbacks=[lgb.early_stopping(stopping_rounds=50)]  
)
```

4. Avaliação e Decisões de Negócio

```
preds_proba = model.predict_proba(X_test)[:, 1]  
print(f"ROC-AUC: {roc_auc_score(y_test, preds_proba):.4f}")
```

5. Explicabilidade (O que gerou a predição?)

```
explainer = shap.TreeExplainer(model)  
shap_values = explainer.shap_values(X_test)  
# shap.summary_plot(shap_values, X_test) # Mostra as features mais importantes visualmente
```

11. STACK TECNOLÓGICA SUGERIDA

Para tirar este modelo do notebook e transformá-up num produto real:

- **Processamento de Dados:** Polars (se couber na memória RAM/Servidor) ou PySpark (se estivéssemos falando de petabytes de dados transacionais ao vivo).
- **Modelagem:** LightGBM + Scikit-Learn.
- **Rastreamento de Experimentos e Artefatos:** MLflow.
- **Orquestração de Pipeline:** Apache Airflow (Roda o pipeline todos os dias à meia-noite).
- **Feature Store:** Feast (Garante que a feature recency calculada para o treino seja exatamente a mesma lógica servida na predição diária).
- **Deploy (Inference):** FastAPI envolto em um container Docker.

12. EXTRA: ARQUITETURA MLOPS E SISTEMA REAL

1. **Online vs Batch Inference:** Propensão a churn não muda segundo a segundo. Uma arquitetura **Batch** é ideal. O Airflow dispara um job de madrugada que recalcula as features usando dados do D-1 (dia anterior), roda a inferência do modelo para toda a base, e salva as tabelas com os scores de propensão de volta no Data Warehouse.
2. **Sistema Real de Retenção:**
A ferramenta de CRM (ex: Salesforce, Braze, ActiveCampaign) lê essa tabela de scores de manhã. Gatilhos pré-configurados disparam campanhas de e-mail e SMS focados apenas nos clientes da zona de risco recuperável.
3. **Monitoramento de Drift (Evidently AI):**
O que acontece se a Olist mudar sua política de frete grátis? A feature `freight_to_price_ratio` vai mudar de distribuição (*Data Drift*), quebrando a predição. O MLOps Pipeline deve incluir um job semanal que compara a distribuição das features atuais contra as features do dia do treinamento usando o Evidently, alertando o Cientista se houver degradação estatística.

Este é o roadmap arquitetural completo. Ele transforma dados operacionais (orders, payments, reviews) em um ativo estratégico (prevenção à perda de receita) com engenharia robusta e acionabilidade de negócio clara.